

GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY
FACULTY OF COMPUTING

DEPARTMENT OF INFORMATION TECHNOLOGY

Software Quality Assurance

IT3153

Student name	SH Hettiarachchi
Student / index number	D/BIT/24/0057
Degree programme	IT
Date of submission	2026/11/03

1. Test Design and Approach

The suite is a Java Maven project built on Selenium WebDriver and TestNG and organised around the Page Object Model, which keeps page interactions separate from test logic. BaseTest handles browser setup and teardown, LoginPage holds the login actions, and SearchPage holds the search and result-page actions. Credentials are read from environment variables rather than written into the source code.

The search test begins from a successful login. It enters "Software Engineer" in LinkedIn's global search field, submits the query, selects a person or company result and clicks it, then checks the URL of the page that opens. In the recorded run, the first result selected was Thomas Frommel, and the expected and actual profile URLs matched.

2. Test Cases and Observed Results

ID	Description	Steps	Expected result	Actual result	Status
TC-01	Valid login	Open the login page; enter valid credentials; click Sign in.	The user reaches an authenticated LinkedIn page.	Login succeeded and the search box was detected.	PASS
TC-02	Search	Search for "Software Engineer" and submit.	The general search results page loads.	The URL contained /search/results/all and the results page loaded.	PASS
TC-03	Click first result	Find the first person or company result; click it; inspect the destination URL.	The selected profile or company page opens, and its URL corresponds to the selected result.	The Thomas Frommel profile opened. The expected and actual URLs were identical: linkedin.com/in/thomas-frommel/.	PASS
TC-04	Invalid password	Enter a valid username with an incorrect password.	The application rejects the credentials and displays an invalid-login message.	First run: LinkedIn displayed a CAPTCHA/security verification and the message "Your noCAPTCHA user response code is missing or invalid." The test was recorded as inconclusive (skipped). Later run: no CAPTCHA appeared, and the test detected LinkedIn's invalid-credentials message, "Wrong email or password." (see Figures 3, 4, 7 and 8).	PASS

Note: *TC-04 was executed twice. In the first run, LinkedIn's CAPTCHA interrupted the credential-rejection flow, so the result was recorded as inconclusive (skipped). In the later run no CAPTCHA appeared and the invalid-credentials message was detected, so the final status is PASS.*

3. Locators and Rationale

Element	Locator used	Rationale
Username field	input[autocomplete='username webauthn']	Targets the visible username field identified during inspection; a generic email selector matched hidden or duplicate elements.
Password field	input[autocomplete='current-password']	Relies on the password field's autocomplete attribute.
Sign-in button	//button[@type='button' and .//span[normalize-space()='Sign in']]	Targets the button by its visible label and type, which avoids hidden duplicates.
Global search box	input[data-testid='typeahead-input']	Uses the test identifier observed on LinkedIn's global search input.
Person/company result	//a[contains(@href, '/in/') or contains(@href, '/company/')]	Matches profile and company URL patterns. The live page should be inspected to confirm that the first matching link belongs to a result card.

4. Evidence Screenshots

Figures 1 to 4 reproduce the Maven/TestNG console output for the login, search and the two invalid-credentials runs. Figure 5 shows the authenticated LinkedIn feed reached after login, and Figures 6 and 7 were captured automatically by the test code. Figure 8 shows the browser during the later invalid-login run.

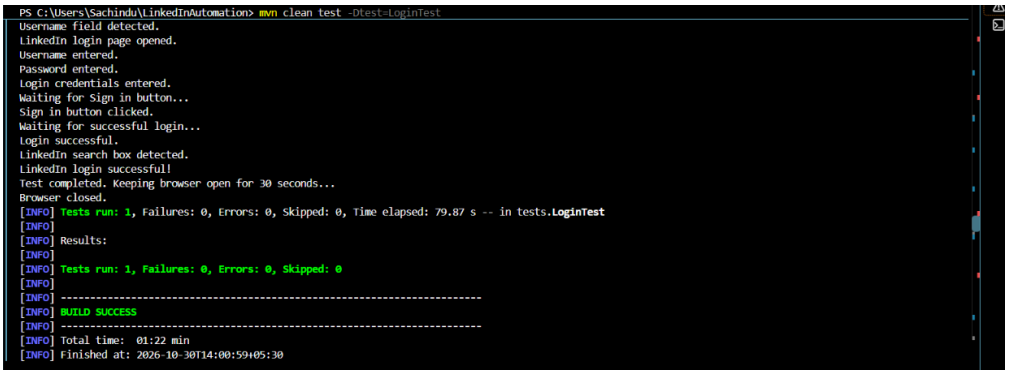


Figure 1. Maven/TestNG console output for the valid-login test (TC-01), ending in BUILD SUCCESS.

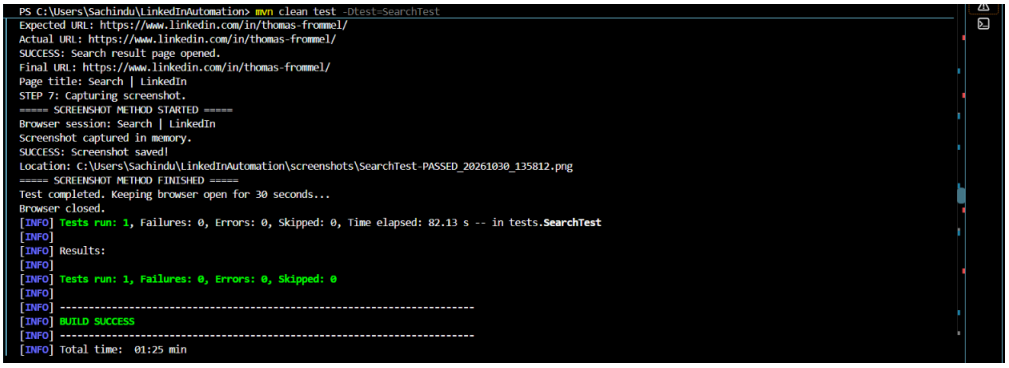


Figure 2. Maven/TestNG console output for the search test (TC-02 and TC-03), ending in BUILD SUCCESS.



Figure 3. Console output for the first invalid-credentials run (TC-04). LinkedIn redirected to a Security Verification page, the CAPTCHA was detected, a screenshot was saved, and Maven reported the test as skipped.

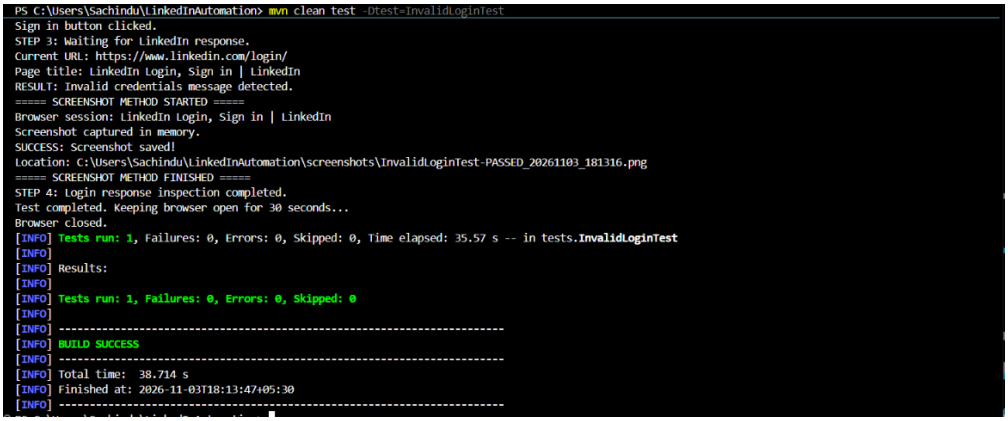


Figure 4. Console output for the later invalid-credentials run (TC-04). The invalid-credentials message was detected, a screenshot was saved, and the test passed.

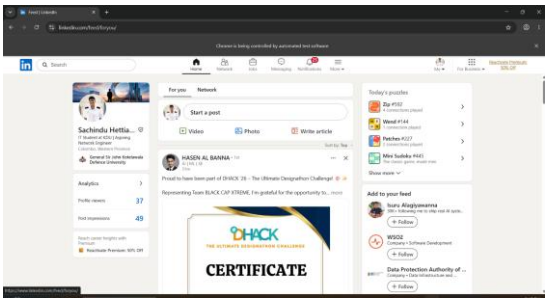


Figure 5. The authenticated LinkedIn feed reached after login (TC-01).

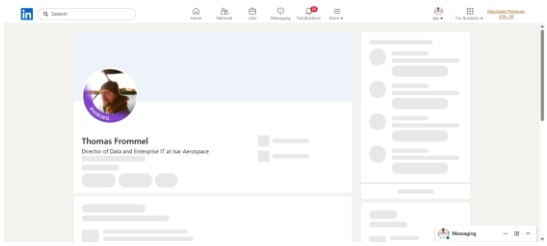


Figure 6. Screenshot captured automatically by the search test (TC-03): the profile page opened from the first result.

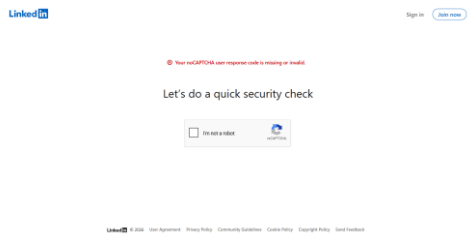


Figure 7. Screenshot captured automatically in the first invalid-login run (TC-04): LinkedIn's security check and the message "Your noCAPTCHA user response code is missing or invalid."

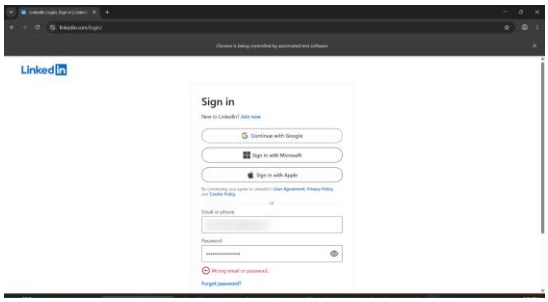


Figure 8. Browser window in the later invalid-login run (TC-04), showing LinkedIn's "Wrong email or password." message. The username has been blurred.

5. Challenges and How They Were Addressed

- **Dynamic elements and stale references.** LinkedIn can replace elements while a page is still loading. To cope with this, the login wait locates fresh elements on each polling cycle and tolerates StaleElementReferenceException, and the search input is relocated during character-by-character entry.
- **Duplicate or hidden login fields.** A broad email-field locator did not reliably identify the visible input, so the implementation uses the observed autocomplete attributes together with a locator for the visible sign-in button.
- **CAPTCHA on invalid login.** LinkedIn sometimes presents a security challenge after an incorrect-password attempt. In the first run, the CAPTCHA response error prevented a reliable check of the normal invalid-credentials message. The challenge was not bypassed, and that run was recorded as inconclusive. In a later run the CAPTCHA did not appear and the test passed.
- **Changing page structure.** LinkedIn is a third-party production site whose markup may change. The locators reflect the DOM observed during testing and may need maintenance if the site is updated.

6. Reflection and Limitations

Thus, the project illustrates how Selenium WebDriver can automate typical UI activities such as signing in, performing search, picking up an option, and validating navigation. However, LinkedIn is a third-party production website and is out of the tester's control. The interface, HTML attributes, authentication logic, anti-bot features, content, and search options may be changed by the site at any time. Search options may also vary across users and sessions. Thus, the successful test only proves what happened during its execution and guarantees nothing about the stability of locators and options in the future.

The CAPTCHA problem showed a limitation of automating production site: the sign-in logic may differ from run to run. In the first case when the login failed, the CAPTCHA did not let validate the expected answer, and the test was marked as inconclusive. However, in the second test run, the automation could detect the message about invalid credentials and the test

succeeded. This proved the importance of dealing with security issues and reporting the correct test results.

While the project provided the automation for all necessary tasks, more reliable negative-login testing should have been done in a controlled test app or in authorized test environment.